

1 Time for Non Shared Memory Gaussian Blur

```
def gaussian(kernel_size=7, sigma=1.0):
    k = kernel_size // 2
    kernel = np.zeros((kernel_size, kernel_size), dtype=np.float32)

    for y in range(-k, k + 1):
        for x in range(-k, k + 1):

            coefficient = 1.0 / (2.0 * np.pi * sigma**2)
            distance_squared = x**2 + y**2
            exponent = -distance_squared / (2.0 * sigma**2)
            gaussian_value = coefficient * np.exp(exponent)
            kernel[y + k, x + k] = gaussian_value

            # Normalize so all weights sum to 1
            kernel /= np.sum(kernel)
    return kernel

kernel = gaussian()

@cuda.jit
def gaussian_blur_kernel(input_img, output_img, kernel):
    x, y = cuda.grid(2)
    height = input_img.shape[0]
    width = input_img.shape[1]
    kernel_size = kernel.shape[0]
    k = kernel_size // 2

    if x < width and y < height:
        r_sum = 0.0
        g_sum = 0.0
        b_sum = 0.0

        for ky in range(-k, k + 1):
            for kx in range(-k, k + 1):
                # Handle edge cases by clamping pixel coordinates
                nx = min(max(x + kx, 0), width - 1)
                ny = min(max(y + ky, 0), height - 1)

                weight = kernel[ky + k, kx + k]

                r_sum += input_img[ny, nx, 0] * weight
                g_sum += input_img[ny, nx, 1] * weight
                b_sum += input_img[ny, nx, 2] * weight
```

```

        output_img[y, x, 0] = np.uint8(r_sum)
        output_img[y, x, 1] = np.uint8(g_sum)
        output_img[y, x, 2] = np.uint8(b_sum)

input_img = img.astype(np.uint8)
height = input_img.shape[0]
width = input_img.shape[1]
d_img = cuda.to_device(input_img)
d_blurred_img = cuda.device_array(input_img.shape, dtype=np.uint8)
d_kernel = cuda.to_device(kernel)

blockSize = (32, 32)
gridSize = (
    (width + blockSize[0] - 1) // blockSize[0],
    (height + blockSize[1] - 1) // blockSize[1]
)
start = time.time()
gaussian_blur_kernel[gridSize, blockSize](d_img, d_blurred_img, d_kernel)
cuda.synchronize()
end = time.time()
output_img = d_blurred_img.copy_to_host()
print(f"Execution time: {end - start:.6f} seconds")

-> Execution time: 0.772504 seconds

```

2 Time for Shared Memory Gaussian Blur

```

kernel_size = 7
radius = kernel_size // 2
blockSize = (32, 32)
mem_size = (blockSize[0] + 2 * radius, blockSize[1] + 2 * radius, radius)
@cuda.jit
def gaussian_blur_shared(input_img, output_img, kernel):
    x, y = cuda.grid(2)
    shared = cuda.shared.array(shape=mem_size, dtype=cuda.uint8)

    tx, ty = cuda.threadIdx.x, cuda.threadIdx.y
    x = cuda.blockIdx.x * blockSize[0] + tx
    y = cuda.blockIdx.y * blockSize[1] + ty

    width = input_img.shape[1]
    height = input_img.shape[0]

```

```

shared_width = blockSize[0] + 2 * radius
shared_height = blockSize[1] + 2 * radius

tid = ty * blockSize[0] + tx
threads_per_block = blockSize[0] * blockSize[1]
total_shared_pixels = shared_width * shared_height

i = tid

while i < total_shared_pixels:

    sy = i // shared_width
    sx = i % shared_width

    # Corresponding global coordinates
    gx = cuda.blockIdx.x * blockSize[0] + sx - radius
    gy = cuda.blockIdx.y * blockSize[1] + sy - radius

    # Clamp image boundaries
    gx = min(max(gx, 0), width - 1)
    gy = min(max(gy, 0), height - 1)

    shared[sy, sx, 0] = input_img[gy, gx, 0]
    shared[sy, sx, 1] = input_img[gy, gx, 1]
    shared[sy, sx, 2] = input_img[gy, gx, 2]

    i += threads_per_block

# Wait until all threads finish loading
    cuda.syncthreads()

# Gaussian convolution
    if x < width and y < height:

        r_sum = 0.0
        g_sum = 0.0
        b_sum = 0.0

        # Position of current pixel in shared memory
        center_x = tx + radius
        center_y = ty + radius

        for ky in range(-radius, radius + 1):
            for kx in range(-radius, radius + 1):

                weight = kernel[

```

```

        ky + radius ,
        kx + radius
    ]

    sx = center_x + kx
    sy = center_y + ky

    r_sum += shared[sy, sx, 0] * weight
    g_sum += shared[sy, sx, 1] * weight
    b_sum += shared[sy, sx, 2] * weight

    output_img[y, x, 0] = np.uint8(r_sum)
    output_img[y, x, 1] = np.uint8(g_sum)
    output_img[y, x, 2] = np.uint8(b_sum)
start = time.time()
gaussian_blur_shared[gridSize, blockSize](d_img, d_blurred_img, d_kernel)
cuda.synchronize()
end = time.time()
print(f"Execution time with shared memory: {(end - start) * 1000:.6f} ms")
output_img = d_blurred_img.copy_to_host()

```

Explanation:

1. Allocate shared memory for kernel
 2. Load pixels from global memory into shared memory
 3. Apply the Gaussian convolution
 4. Copy the result back to the CPU
- > Execution time with shared memory: 671.770811 ms

3 Block size vs Time

- Increase the block size, the time will decrease.

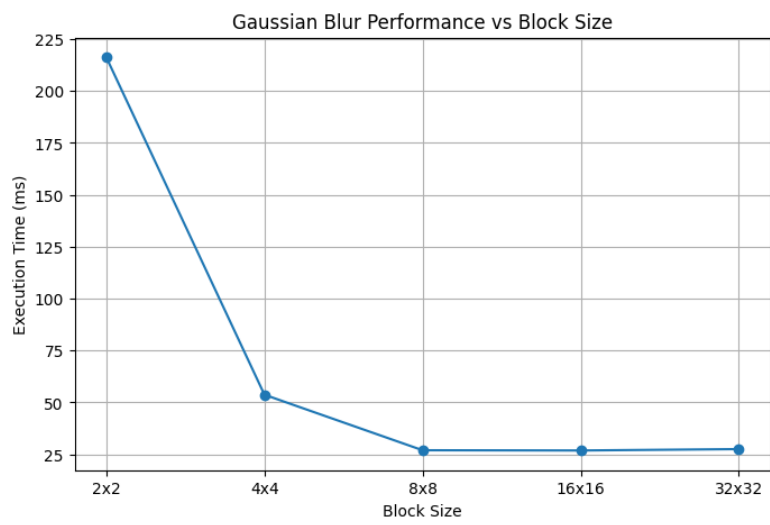


Figure 1: Block size vs Time